21CSE353T · Augmented, Virtual and Mixed Reality

CT-2 Preparation Notes

Unit 4: Unreal Engine & Techniques
Unit 5: Modeling the Physical World

How to use this file. Part 1 is the exam plan. Parts 2 and 3 are the full theory notes for Unit 4 and Unit 5 built from your slides. Part 4 solves the sample paper MCQs and adds a practice bank. Part 5 gives full 10-mark model answers for every Section B question in the sample paper. Part 6 is a last-night revision sheet.

Part	Content
1	Exam pattern, mark plan, how to answer
2	Unit 4 notes: Unreal vs Unity, Chaos physics, optimization, applications, XR pipeline, future trends, lab steps
3	Unit 5 notes: geometric, kinematic, physical modeling, locomotion, Quest setup, UI/UX, audio/video, metaverse, behavior
4	Sample paper Section A answers with reasons + extra MCQ practice bank
5	Section B model answers (21a, 21b, 22a, 22b, 23a, 23b)
6	One-page revision sheet, numbers to remember, viva questions

Note on the schedule. The pattern you pasted said CT-1 / Unit 1 & 2, but the slides and sample paper you uploaded are Unit 4 & 5 (CT-2, dated 10/11/2026). These notes follow the uploaded files. The paper format is identical: 20 MCQs (6 from Unit 4, 14 from Unit 5) + 3 either-or questions (1 from Unit 4, 2 from Unit 5).

Part 1 – Exam Plan

Item	Detail
Part A	20 MCQs x 1 mark = 20. Unit 4 = Q1–6 (CO4), Unit 5 = Q7–20 (CO5)
Part B	3 either-or questions x 10 = 30. Q21 from Unit 4, Q22 and Q23 from Unit 5
Duration / Total	100 min / 50 marks (sample paper)
Bloom's level	40% of marks are L4 (Analyze). Expect scenario questions, not definitions

What the sample paper tells you

- Every Part B question is a **real-world scenario** with two sub-parts (1 and 2), about 5 marks each. Answer both parts, label them 1 and 2.
- Q21 (Unit 4) always compares Unreal vs Unity or asks about Unreal features (Blueprints, C++, Nanite, Lumen, LOD, occlusion culling, OpenXR, cloud rendering, AI content).
- Q22/Q23 (Unit 5) mix modeling types (geometric, kinematic, physical, behavior) with locomotion, Guardian, grab interaction, world-space UI, spatial/ambisonic audio, 360 video, metaverse.
- Part A is mostly direct from slides. 14 of 20 MCQs are Unit 5, so Unit 5 is the bigger scoring area.

10-mark answer template (works for every scenario question)

- **Line 1:** one-sentence restatement of the scenario's need (shows you understood).
- **Part 1:** 4–5 points. Each point = term + what it does + how it applies to *this* scenario.
- **Part 2:** 4–5 points, ending with a justification sentence (safety, comfort, performance, immersion).
- Add a small comparison table if the question says 'compare'. Use slide keywords: Lumen, Nanite, Blueprints, Chaos, HRTF, vignette, Guardian, diegetic, FSM, NavMesh.
- Always link back to **90 fps / 72 fps, cybersickness, immersion** where relevant.

Part 2 – Unit 4: Unreal Engine & Techniques

Syllabus map (CO-1 to CO-5): Unreal vs Unity; Unreal physics (rigid body, collision, cloth, fluids/destruction); optimization (LOD, occlusion culling, draw-call batching, profiling); application demos; future of MR/xR.

4.1 Unreal Engine 5 vs Unity 6

Feature	Unreal Engine 5	Unity 6
Language	C++ and Blueprints (node-based visual scripting)	C#
Rendering	Lumen (dynamic global illumination) + Nanite (virtualized geometry)	HDRP / URP pipelines
Target use	AAA games, film, architectural visualization (archviz), high-fidelity VR	Mobile, AR, indie, XR, rapid prototyping
AR/VR SDK	OpenXR, SteamVR, Meta plugin	AR Foundation, XR Interaction Toolkit
Asset pipeline	FBX/USD, Megascans built in	Asset Store, FBX import
Physics engine	Chaos Physics (built in)	PhysX / custom
Licence	Free + 5% royalty above $1M revenue	Free / Pro tiers
Multiplayer	Built-in replication / networking framework	Netcode for GameObjects (package)

Why choose Unreal (use in 'justify' answers)

- **Fidelity:** Lumen gives real-time GI and reflections with no baked lightmaps; Nanite handles film-quality meshes without hand-made LODs.
- **Workflow:** Blueprints let non-programmers (designers) iterate; C++ is available for performance-critical systems.
- **VR tooling:** OpenXR support, Unreal Insights, stat commands, instanced static meshes, occlusion culling.
- **Best for:** archviz, AAA, film, high-end VR. **Unity is better for:** mobile AR (Pokémon GO, IKEA Place), HoloLens medical training (MRTK), indie and quick prototypes.

4.2 Industry examples

Engine	Example	Why
Unreal	Fortnite / The Matrix Awakens demo (2021)	Lumen GI, film-quality visuals at 30 fps on PS5/Xbox
Unreal	Architectural visualization: BMW, IKEA, Zaha Hadid Architects	Real-time walkthroughs replace costly CGI renders; clients change materials live
Unity	Pokémon GO / IKEA Place AR	AR Foundation + ARCore; low overhead for mobile GPUs
Unity	Microsoft HoloLens medical training	MRTK gives hand tracking, voice commands, spatial anchors; surgeons rehearse on holographic anatomy

4.3 Unreal Chaos Physics

Component	How it works
Rigid body dynamics	Fixed-shape objects respond to forces: gravity, impulse, torque. Each actor has a physics body with mass, damping and collision response settings.
Collision detection	Simple collision (boxes, spheres, capsules) for physics and complex collision (actual mesh) are separate. Pipeline: **broadphase → narrowphase → contact resolution**.
Cloth simulation	Chaos Cloth attaches cloth meshes to skeletal actors. Wind velocity, stiffness, damping set per panel. Used for capes, flags, curtains.
Fluid and destruction	Chaos Destruction fractures meshes into procedural fragments using **Voronoi decomposition**. Field effects (radial force, noise) drive destruction on impact.

4.4 Optimization techniques (target 90 fps for VR)

Technique	Engine	What it does	Unreal / Unity tool
Level of Detail (LOD)	Both	Swaps distant meshes for lower-poly versions. Saves roughly 40–60% GPU cost at distance	Auto LOD generation / LOD Group

Technique	Engine	What it does	Unreal / Unity tool
Occlusion culling	Both	Skips rendering objects hidden behind others (e.g. behind a wall). Critical for indoor VR	Precomputed + dynamic occlusion / Umbra baking
Draw call batching	Both	Combines many small meshes into one GPU call. Vital for foliage and crowds	Instanced Static Mesh / GPU Instancing + Static Batching
Nanite	Unreal (UE5)	Virtualized micro-polygon geometry; film-quality assets without poly-count worry; streams detail automatically	Nanite
Lumen	Unreal	Fully dynamic global illumination, soft shadows, no baking; enables live lighting change and day/night cycles	Lumen
Profiling	Both	Find the CPU/GPU bottleneck first, then optimize	Unreal Insights, stat fps / stat GPU / stat unit; Unity Profiler, Frame Debugger

Exam tip: 'Which technique skips objects not visible to the camera?' = **Occlusion culling**. 'Film-quality meshes without manual LODs?' = **Nanite**. 'Dynamic GI without lightmaps?' = **Lumen**. 'Many copies in one draw call?' = **Instancing / batching**.

4.5 How Unreal powers an XR application (pipeline)

Stage	Component	Detail
1	Real-world sensors	Camera, LiDAR, depth, IMU. Depth camera maps space at 30 Hz
2	Tracking and mapping	ARCore / ARKit, 6DoF SLAM, 3D point cloud
3	Unreal Engine core	Physics, rendering, Blueprint logic; spawns objects at tracked anchors; physics at 120 Hz
4	XR display output	HMD / phone / projector / HUD at 90 fps with head-pose warp

Feedback loop: hand tracking or controller input → Unreal event system → updates the simulation → new frame.

4.6 Unreal in the real world (six verticals)

Vertical	Example	Detail
Gaming	Fortnite / PUBG	Nanite + Lumen render 1M-polygon environments at 60 fps with dynamic GI
Architecture	BMW, IKEA, Zaha Hadid	Real-time walkthroughs replacing $50K CGI renders; live material switching
Medical training	Osso VR Surgical Sim	Haptic glove + Unreal simulates bone-cutting resistance
Military / defence	US Army IVAS (HoloLens 2)	Mixed-reality HUD with terrain overlays
Industrial MR	Ford assembly line AR	Holographic assembly instructions; reduces errors by 30%
Education	Google Expeditions / ENGAGE	VR lectures with spatial audio and hand tracking

4.7 Future of MR / xR (next 5 years)

Year	Trend	Key point
2024	Spatial computing	Apple Vision Pro + Unreal visionOS plugin; passthrough AR; persistent room-scale anchors
2025	AI-driven avatars	MetaHuman + GPT-4 voice; digital humans with real-time LLM dialog; virtual receptionists, training
2026	6G edge rendering	Ultra-low-latency cloud rendering offloads GPU to edge servers; thin HMDs stream photo-real XR
2027	Haptics and sensorics	Full-hand haptic gloves (HaptX/Ultraleap) + Unreal physics: feel texture, weight, impact
2028	Neural interfaces	BCI headsets read intent; Unreal responds in under 10 ms; accessibility-first non-controller apps

Present limitations to mention in 21b: heavy GPU load on standalone headsets, fragmented device SDKs, large build/package sizes, battery and thermal limits, high latency without edge compute, manual content creation cost. **Directions:** OpenXR standardization, cloud/edge rendering, AI-assisted content creation, haptics, BCI.

4.8 Hands-on lab summary (steps that can come as 'list the steps')

Install Unreal Engine 5

- Windows 10/11 64-bit, 8-core CPU, 32 GB RAM, RTX 2060, 80 GB SSD.
- Download Epic Games Launcher → sign in/create account → Library → '+' → choose UE 5.3 or latest → Install → Launch (Project Browser appears).

Configure XR (works with Quest 2/3, Vive, Index, HoloLens 2, WMR)

- Project Browser → Games → Blank → Desktop/Console → Raytracing OFF → name project → Create.
- Edit → Plugins → enable **OpenXR** (+ OpenXR Tools), plus OculusVR or SteamVR; restart.
- Project Settings → Input → Use Enhanced Input; Plugins → OpenXR → Enable; set stereo rendering to OpenXR.
- Connect headset (USB/Link) → Play dropdown → **VR Preview**.

Blueprint: grab and throw a physics cube

- Place Cube; collision = Physics Actor; tick Simulate Physics; mass 2 kg.
- Open Blueprint editor (Event Graph, 'Event BeginPlay').
- Add motion-controller input node (Grip Pressed). On grip: *Attach Component To Component* (Snap to Target); on release: *Detach From Component*.
- Compile → Save → Play in VR Preview.

Chaos destruction: shatter a wall

- Enable Chaos Physics + Geometry Collection plugins; restart.
- Select wall → Fracture → Voronoi → Uniform, cell count 30 → Fracture (creates a Geometry Collection).
- Damage Threshold = 100 (fractures at 100+ impulse units).
- Add Radial Falloff Field: strength 500000, radius 200 cm.
- Ball Blueprint: On Hit → field component → Apply External Strain.

Optimize and package

- Console (~): stat fps, stat GPU, stat unit. Red frame time = bottleneck.
- Auto LOD generation (LOD1 at 50% triangles, LOD2 at 25%).
- Convert repeated props to Instanced Static Mesh (e.g. 200 draw calls into 1).
- Project Settings → Rendering → Occlusion Culling ON (hardware query for dynamic).
- File → Package Project → Android (Quest) or Windows. Development build first, Shipping for release. Install with *adb install* or Meta Dev Hub.

Unity AR Foundation (mobile AR)

- Unity 6 LTS; Package Manager → AR Foundation + ARCore XR Plugin (Android) or ARKit XR Plugin (iOS).
- Build Settings → Android; Minimum API Level 24; XR Plug-in Management → tick ARCore.
- Add AR Session + AR Session Origin (replaces Main Camera) for camera pose tracking.
- Add **AR Plane Manager**; script raycasts touch position against detected planes and Instantiates a prefab at the hit pose.
- Build And Run on phone (USB debugging on). Point at floor, blue plane mesh appears, tap to place object.

Part 3 – Unit 5: Modeling the Physical World

Session plan: Hour 1 modeling the world (geometric, kinematic, physical). Hour 2 being in the world (locomotion, Quest setup/interaction, UI/UX, audio/video). Hour 3 sharing the world (metaverse, behavior modeling).

5.0 The VR modeling pipeline

Model	Question it answers	Example
Geometric	What does it **look** like?	Mesh, textures, materials
Kinematic	How does it **move**?	Position, rotation, joints, no forces
Physical	How does it **react**?	Mass, gravity, collision, friction, haptics
Behavior	How does it **decide**?	Rules, FSM, AI, NPCs

Kinematics says *where* an object goes. Physics explains *why* it goes there.

5.1 Geometric modeling

Describes the shape and appearance of every virtual object: **object = shape + appearance**.

- **Shape:** vertices in 3D object space, edges, polygonal faces, topology, bounding volume for fast tests.
- **Appearance:** material and colour, textures via UV coordinates, shading, transparency, reflections.
- **Sources:** Blender/Maya, 3D scanning, photogrammetry, LiDAR, procedural generation, asset stores.

Polygon meshes: the VR workhorse

- **Vertex:** point with position, normal, UV and colour. **Edge:** line joining two vertices. **Face:** a triangle; every quad becomes two triangles on the GPU.
- **Why triangles?** Three points are always planar so the GPU shades them fast. A standalone headset renders each frame twice (once per eye), so every extra triangle costs double.

Representation	How shape is stored	Use
Polygon mesh	Vertices joined into triangles	Nearly all real-time VR and games
NURBS / splines	Control points defining smooth curves	CAD, car and aircraft bodies (meshed before VR)
CSG	Boolean union, subtract, intersect of primitives	Level blocking, mechanical parts
Voxels	3D grid of small cubes	Minecraft-style worlds, CT/MRI scans
Point cloud / splats	Millions of measured surface points	LiDAR scans, digital twins

PBR materials

Map	Purpose
UV mapping	Unfolds 3D surface onto a flat 2D image (like opening a cardboard box)
Albedo	Base colour with no baked lighting
Normal map	Fakes bumps/scratches without adding triangles
Metallic map	Marks metal vs plastic/wood regions
Smoothness	Sharp reflections (steel) vs soft (concrete)
Occlusion map	Darkens creases and corners

Blender to Quest in six steps

- 1. Model/scan in Blender, apply scale so 1 unit = 1 metre. 2. Reduce and unwrap: decimate, UV unwrap, bake detail into a normal map.
- 3. Export FBX or glTF with textures. 4. Import to Unity, check scale factor, assign URP Lit materials.
- 5. Add LOD Group (LOD0/1/2) and bake occlusion culling. 6. Profile on headset; hold 72 fps or higher.
- **Most common bug:** model imported 100x too big/small from a centimetre vs metre mismatch.

Real example: virtual heritage temple tour: capture (drone/phone photos) → reconstruct (photogrammetry dense mesh) → optimize (decimate, bake normals) → assemble (Unity, LODs, baked light) → deploy on Quest. Decimation and normal-map baking fix the weight; the loss is fine geometric detail.

5.2 Kinematics modeling

Position, orientation and motion of objects **without considering forces**.

- **Pose** = where an object is and which way it faces. **6 DoF** = 3 translations (X left/right, Y up/down, Z forward/back) + 3 rotations (pitch = nod, yaw = shake 'no', roll = tilt head).
- Quest headset and controllers track all 6 DoF; a phone VR viewer tracks only 3 (rotation).
- **Homogeneous transform:** a 4x4 matrix = 3x3 rotation + translation (tx, ty, tz) with bottom row 0 0 0 1. Moves combine by multiplication: **p_world = T · R · S · p_local**.
- **Scene graph:** parent-child hierarchy. M_world(child) = M_world(parent) · M_local(child). Move the parent and every child follows. Grabbing = re-parenting the object to the hand.

	Forward kinematics (FK)	Inverse kinematics (IK)
Input → output	Joint angles → hand position	Target hand position → joint angles
Solutions	One answer, easy to compute	Many or none; solved numerically
Formula	x = L1 cos(t1) + L2 cos(t1 + t2)	Iterative solvers
In VR	Animating a known arm pose	Only head and hands are tracked, so IK estimates elbows, shoulders, spine for a full-body avatar

Unity: kinematic door on a hinge

- Script uses **transform.localRotation** (relative to parent frame), **Quaternion.Slerp** to blend smoothly, and **Time.deltaTime** so speed is the same at 72 or 120 fps.
- Other tools: Animator (keyframed motion), Animation Rigging's Two Bone IK for arms/legs.
- Real example: VR teleoperation of a robot arm: Quest controller pose (6 DoF) → IK solver computes six joint angles → Unity twin moves and angles sent to the real arm via ROS-TCP connector. If the target is out of reach, the IK solver should clamp to the nearest reachable pose and the UI should show a warning/colour change.

5.3 Physical modeling

Gives objects physical properties: mass, gravity, collisions, friction, deformation and touch feedback.

Aspect	Meaning
Collision detection	Are two objects touching or overlapping now?
Dynamics and response	Newton's laws: mass, gravity, velocity, bounce
Surface and deformation	Friction, elasticity, rigid vs soft bodies (cloth, tissue)
Force and haptic feedback	Vibration or resistance so the user feels contact

Collision detection in two phases

- **Broad phase:** wrap objects in simple bounding volumes (AABB, bounding sphere, OBB, BVH trees, spatial grids) and quickly discard far-apart pairs.
- **Narrow phase:** exact tests on the few remaining pairs (separating axis test, GJK) to find contact points, normal and depth.
- **Response:** push objects apart, apply bounce and friction, fire events (sound, haptics).
- Unity: Box, Sphere, Capsule colliders are cheap; Mesh Collider is accurate but costly and must be convex to move with a Rigidbody.

One physics step every frame

- 1) Sum forces (gravity, pushes, springs). 2) a = F/m; v += a·dt; x += v·dt. 3) Detect collisions at new positions. 4) Resolve with bounce (e) and friction (μ).
- Unity runs this in **FixedUpdate** at a fixed time step (default 0.02 s); match to headset refresh rate for smooth hand-held objects.

Unity setting	Physical meaning
Mass	Inertia; resists being pushed
Drag	Air resistance slowing motion
Use Gravity	Falls at 9.81 m/s2
Is Kinematic	Moved by script/animation, not by forces
Bounciness	Restitution: 0 = dead, 1 = perfect bounce

Unity setting	Physical meaning
Friction	Grip between surfaces
Joints	Hinge, spring, fixed constraints

Throwable ball example: Sphere scale 0.22, Rigidbody mass 0.43, Continuous collision detection, Physic Material bounciness 0.6, XR Grab Interactable with Velocity Tracking and Throw On Detach, Audio Source + script using OnCollisionEnter and c.relativeVelocity.magnitude to scale volume. Velocity Tracking moves the held ball with physics so it still collides with walls in your hand.

Fire-safety training example: lift extinguisher = mass + two-hand grab; pull pin = fixed joint breaking at a force; spray = particles with colliders; feel spray = continuous controller vibration; fire shrinks as particle hits reduce intensity. Realistic physics improves training transfer.

5.4 Locomotion tools

Problem 1 (space): Guardian boundary limits real walking (e.g. 3 m room vs a 2 km virtual campus). Problem 2 (sensory conflict): eyes see motion (vection) but the inner ear feels none, which causes **cybersickness**.

Technique	How it works	Comfort
Room-scale walking	Physically walk inside the boundary	Highest
Teleportation	Point an arc, blink to landing spot (loses motion continuity)	High
Snap turn	Rotate in fixed steps (30 or 45 degrees)	High
Smooth movement	Glide with thumbstick like a game	Low to medium
Grab-move / climbing	Pull the world past you with hands	Medium
Arm-swinging	Swing arms like jogging to move	Medium
Redirected walking	Subtly rotate the world to keep you in the room	High, needs space
Vehicle / cockpit	Seated in car, plane, wheelchair	Medium

Six rules for comfortable motion

- 1. **Tunnelling vignette**: darken peripheral vision while moving. 2. **Snap, not smooth, turns**: rotation is the strongest sickness trigger.
- 3. **Stable reference**: cockpit, dashboard or horizon that moves with you. 4. **No surprise acceleration**: constant speed; never move the camera without the user.
- 5. **Hold frame rate**: 72/90/120 Hz with low motion-to-photon latency. 6. **Let users choose**: comfort settings, seated mode, adjustable speed.

Unity XR Interaction Toolkit: import XRI Starter Assets → XR Origin (XR Rig) prefab (Camera Offset → Main Camera, Left/Right Controller, Locomotion with Teleportation Provider, Snap Turn Provider, Continuous Move Provider, Tunneling Vignette) → Teleportation Area on floor (needs a collider) → Teleportation Anchors at fixed viewpoints → snap angle 45 degrees, speed about 2 m/s → vignette + comfort toggle.

5.5 Oculus (Meta Quest) setup and object interaction

Feature	Detail
Inside-out tracking	Headset cameras track 6 DoF; no external sensors
Controllers + hands	Touch controllers with haptics, or camera-based hand tracking
Colour passthrough	See the real room for mixed reality apps
Chip	Snapdragon XR2 Gen 2; up to 120 Hz on Quest 3
OS	Horizon OS, Android-based, so Unity builds an Android APK
Boundary (Guardian)	User-drawn safe play area, stationary or room-scale; prevents collisions with real obstacles
2026 lineup	Quest 3 (flagship), Quest 3S (budget); Quest 2 and Pro still usable for labs

Unity + Quest setup in eight steps

- 1) Install Unity 6 or 2022.3 LTS with Android Build Support. 2) Turn on Developer Mode in the Meta Horizon phone app.

- 3) Connect USB-C and allow USB debugging in the headset. 4) Create project from VR or Mixed Reality template.
- 5) Build Settings: switch platform to Android. 6) XR Plug-in Management: OpenXR + Meta Quest feature group.
- 7) Add the Oculus Touch controller interaction profile. 8) Project Validation → Fix All → Build And Run.
- Key packages: XR Interaction Toolkit, XR Hands, OpenXR: Meta (or Meta XR SDK for Meta-only features).

Interactors meet interactables (XR Interaction Manager decides who can touch what)

Interactors (the hand)	Interactables (the object)	States
Direct: grab what you touch Ray: point at far objects Poke: press with fingertip Socket: slot that holds objects	Grab: pick up and throw Simple: events only (button) UI: world-space canvases	Hover → highlight Select (grip) → pick up Activate (trigger) → use

- **Torch example:** Torch model + Rigidbody + Collider, XR Grab Interactable with Attach Transform at the handle, Spot Light child; script listens to *activated* (toggle light) and *selectExited* (light off).
- **Hand tracking:** 26 tracked joints per hand in OpenXR via XR Hands package. Gestures: **Pinch** (thumb+index select, works with ray), **Poke** (fingertip press), **Grab** (close hand). Bare hands feel no vibration, so use big targets with clear visual and audio feedback.

5.6 UI/UX on Meta Quest

	Part of the story	Not part of the story
In the 3D world	**Diegetic (best):** wristwatch, car dashboard, tablet the user holds	**Spatial (good):** floating panel, labels above exhibits, wrist menu
Not in the 3D world	**Meta (use lightly):** vision goes red when hurt	**Non-diegetic (avoid):** HUD glued to the screen

Where to place UI

- **Distance:** about 1–3 m; nothing reading-heavy closer than 0.5 m. **Angle:** key content within about ±30 degrees of straight ahead, slightly below eye level.
- **Curve** wide panels so every part is equidistant. **Follow:** lazy-follow or world-locked, not glued to the head. **Text:** large, bold, high contrast.
- **Why not screen-space UI?** It breaks immersion and causes discomfort since the head and eyes move freely in 3D.

Building a world-space menu in Unity

- Canvas Render Mode = **World Space**; scale 0.001 (1,000 UI pixels = 1 m); place 1.5 m ahead.
- Replace Graphic Raycaster with **Tracked Device Graphic Raycaster**; EventSystem uses **XR UI Input Module**; ray interactor for far panels, poke for near ones.
- **Wrist menu pattern:** small canvas parented to left hand/controller, shown only when the palm faces the face, right hand pokes/points. Always available, never blocks view.

Good VR UX = see it, hear it, feel it: visual (glow on hover, press depth), audio (click, whoosh, confirm tone), haptic (short pulse on grab/press). Design for understanding (affordances, short onboarding, consistent buttons) and for everyone (seated mode, height reset, one-handed play, subtitles, colour-blind-safe signals).

5.7 Audio and video in VR

Spatial audio

Cue	Meaning
ITD	Interaural time difference: nearer ear hears it first
ILD	Interaural level difference: head shadows the far ear
HRTF	Head-related transfer function: ear shape filters sound, tells up/down/front/back

- Unity: Audio Source **Spatial Blend = 1 (3D)**, logarithmic rolloff with min/max distance, Spatializer plugin for HRTF (e.g. Meta XR Audio), reverb zones, Audio Listener on the head camera.
- **Ambisonic audio:** full-sphere surround sound that can be rotated to match the listener's head orientation; used with 360 video. **Purpose of spatial audio:** directional and distance cues matching

the source position.

360 degree video

Format	What the viewer gets
360 mono	Look all around; flat, no depth
360 stereo	All around with depth; two views stacked
180 stereo	Front half only, sharper, with depth
Volumetric	Captured 3D people you can walk around

- 360 video is **3 DoF**: you can turn but leaning does not change the view. Use high resolution and hardware-decoded H.264/H.265.
- Unity pipeline: Video Player → Render Texture → Skybox/Panoramic material (360) or a quad screen (virtual cinema).
- Difference from flat video: it surrounds the viewer and lets them look in any direction during playback.

5.8 Introduction to the metaverse

Definition: a network of persistent, shared, real-time 3D spaces where people meet as avatars, create, work and trade. Term from Neal Stephenson's 1992 novel *Snow Crash*.

- **Properties:** persistent (continues after you log off), synchronous (events seen live), shared (many users, one world), embodied (avatar), creator economy, interoperable (still unmet goal).
- **Seven layers (Jon Radoff):** 7 Experience, 6 Discovery, 5 Creator economy, 4 Spatial computing (3D engines, physics, AR/VR: this course), 3 Decentralization, 2 Human interface, 1 Infrastructure.
- **Uses:** education (virtual labs), healthcare (surgical rehearsal), industry (digital twins), social/events (Roblox, Fortnite concerts), retail/real estate, workplace.
- **2026 reality check:** Meta's Reality Labs has lost about $80B since 2020; in March 2026 Horizon Worlds was moved toward mobile-first and focus shifted to AI and smart glasses. Growth continues in digital twins, training, mass virtual worlds on phones/PCs and mixed reality. Lesson: hype vs real value.

5.9 Behavior modeling

Objects and characters that **sense, decide and act** on their own. Sense (is the user near? what did they say?) → Decide (rules, state machines, trees, learned policies) → Act (move, animate, speak, change the world).

Level	Type	Example
1	Reactive	Door opens as you approach
2	Time-driven	Traffic lights cycle, day turns to night
3	Agent	Guide NPC with goals and states
4	Intelligent	Learns or converses using AI

Technique	Core idea	VR example
Event rules	If event then action	Alarm when valve left open
State machine	Fixed states with transitions	Guard: patrol, chase, return
Behaviour tree	Prioritised tree of tasks	Shopkeeper serving queues
NavMesh steering	Path-finding on walkable areas	Crowds avoiding the user
Utility AI	Score every option, pick best	Pet choosing eat, play or sleep
Reinforcement learning	Learn from rewards (ML-Agents)	Robot learning to grasp
LLM-driven	Language model plans and talks	Tutor answering questions

- **Campus guide FSM:** Idle (waits at gate) → [user within 3 m] Greet → [user asks for tour] Guide (walks to next stop) → [arrived] Explain → [more stops left] back to Guide, or [tour over / user leaves] back to Idle. One state at a time; each arrow is a condition checked every frame.
- **Unity:** mark floors Navigation Static and bake a NavMesh, add NavMeshAgent to the guide, empty objects as tour stops, enum State + switch in Update, agent.SetDestination(stops[next].position).
- **Typical exam example (Q20):** a virtual shopkeeper that greets, answers questions and restocks shelves when inventory runs low = behavior modeling.

Layer (campus open day)	What we build	Unity tool
Geometric	Scanned campus blocks, labs	FBX import, LOD Group
Kinematic	Lift doors, avatar arms via IK	Transforms, Animation Rigging
Physical	Grabbable apparatus, solid walls	Rigidbody, colliders
Locomotion	Teleport anchors per building	XRI Teleportation
UI/UX	Wrist map menu, info panels	World-space canvas
Audio/video	Guide's 3D voice, 360 hostel clips	Spatial audio, Video Player
Multi-user	Parents and students tour together	Netcode for GameObjects
Behavior	Guide NPC FSM, AI FAQs	NavMesh, FSM script

Part 4 – Part A (MCQ) Practice

4A. Sample paper Section A: answers with reasons

Q	Answer	Reason
1	A) Blueprints	Node-based visual scripting in Unreal. Shader Graph is Unity's material tool; Nanite is geometry.
2	B) C++	Unreal's native language; Unity uses C#.
3	B)	Nanite = film-quality detailed meshes in real time without manual LOD versions.
4	A) Occlusion culling	Skips objects hidden from the camera view.
5	B)	Lumen = fully dynamic global illumination and reflections, no baked lightmaps.
6	B)	Cross-device standardization (OpenXR), cloud/edge rendering and AI-assisted content.
7	B)	Geometric modeling = mathematical representation of shape, surface boundaries and structure.
8	A)	Kinematics = position, velocity, angles of motion without considering forces.
9	A)	Physical modeling adds forces, mass, friction and collision response.
10	B) Teleportation	Instant move, reduces cybersickness but loses motion continuity.
11	B)	Guardian prevents collisions with real-world obstacles during room-scale play.
12	B) Grab/Grip	Closing fingers/controller grip around an object picks it up.
13	B)	Spatial audio gives directional and distance cues matching the source.
14	B)	Metaverse = persistent, interconnected shared 3D spaces with avatars, social and economic activity.
15	B)	Behavior modeling defines how an entity reacts and moves over time to events, input and rules.
16	A)	360 video surrounds the viewer and allows free looking in any direction.
17	B)	Passthrough shows a live video feed of the real surroundings while wearing the headset.
18	B)	Screen-fixed UI breaks immersion and causes discomfort; world-space UI is used instead.
19	A)	Ambisonic = full-sphere surround sound rotatable with head orientation.
20	B)	Shopkeeper that greets, answers and restocks = behavior modeling.

4B. Extra practice bank (answers in brackets)

Q. Unreal's collision pipeline order? → Broadphase, narrowphase, contact resolution.

Q. Technique that breaks meshes into fragments using Voronoi? → Chaos Destruction (Geometry Collection fracture).

Q. Unreal's built-in physics engine? → Chaos Physics (Unity: PhysX).

Q. Which engine is more used for mobile AR such as IKEA Place? → Unity with AR Foundation + ARCore.

Q. Target frame rate for comfortable VR in Unit 4 slides? → 90 fps (Quest labs in Unit 5: keep 72 fps or higher).

Q. Console commands to profile Unreal? → stat fps, stat GPU, stat unit; also Unreal Insights.

Q. Unreal licence model? → Free, 5% royalty above $1M revenue.

Q. Unity minimum Android API level for ARCore (slide)? → 24.

Q. Which Unity component detects floors for AR? → AR Plane Manager.

Q. How many degrees of freedom do Quest headset/controllers track? → 6 DoF (phone VR viewer: 3).

Q. Matrix size for homogeneous transform in 3D? → 4x4.

Q. Which Unity function runs physics at fixed step? → FixedUpdate (default 0.02 s).

Q. What does Is Kinematic do? → Object moved by script/animation, not by forces.

Q. Unity method for smooth rotation blending? → Quaternion.Slerp.

Q. Which collider is expensive but accurate? → Mesh Collider (must be convex to move with Rigidbody).

Q. Strongest cybersickness trigger in locomotion? → Rotation (so use snap turn).

Q. Effect of a tunnelling vignette? → Darkens peripheral vision during movement to reduce sickness.
Q. OS of Meta Quest and what Unity builds? → Horizon OS (Android-based), Android APK.
Q. Interactor that presses with fingertip? → Poke interactor.
Q. Event on trigger press in XRI? → Activate (hover = highlight, select = grip pickup).
Q. Number of tracked joints per hand (OpenXR)? → 26.
Q. Best UI type in VR? → Diegetic UI; avoid non-diegetic HUDs.
Q. Recommended UI distance? → About 1 to 3 m; nothing reading-heavy closer than 0.5 m.
Q. Audio cue from head shadowing? → ILD (level difference). ITD = time difference.
Q. Spatial Blend value for fully 3D sound in Unity? → 1.
Q. 360 video degrees of freedom? → 3 DoF.
Q. Who coined the term metaverse and where? → Neal Stephenson, Snow Crash (1992).
Q. Which layer of the metaverse is spatial computing? → Layer 4.
Q. Unity package/tool for NPC path-finding? → NavMesh / NavMeshAgent.
Q. Behavior technique with fixed states and transitions? → Finite state machine.

Part 5 – Section B Model Answers (10 marks each)

Write each answer with the two labeled parts. These are about the length that fits in 25–30 minutes. Attempt any 3 of the 3 pairs (one from each either-or).

Q21 (a) Unreal vs Unity for a photorealistic VR architectural walkthrough

1. Comparison and justification

Criterion	Unreal Engine 5	Unity 6
Rendering fidelity	Lumen real-time GI and reflections, Nanite for dense geometry, Megascans built in; film-quality out of the box	HDRP can reach high fidelity but needs more setup; no native equivalent of Nanite
Scripting / visual scripting	Blueprints (node-based) let non-programmer designers build logic; C++ for performance-critical parts	C# scripting; visual scripting is a separate package and less central
VR optimization tooling	OpenXR, VR Preview, Unreal Insights, stat commands, instanced static mesh, occlusion culling, auto LOD	XR Interaction Toolkit, Profiler, Frame Debugger, GPU Instancing, LOD Group, Umbra occlusion
Typical strength	AAA, film, archviz, high-fidelity VR	Mobile AR, indie, rapid prototyping

Justification: the studio needs Lumen lighting, Nanite detail and designer-friendly Blueprints, which are native Unreal features. Archviz firms such as BMW, IKEA and Zaha Hadid already use Unreal for real-time walkthroughs where clients swap materials live. Unity would need extra pipelines and custom work to match this, so Unreal is the more suitable choice.

2. Two techniques that keep the scene at 90 fps

- **Nanite virtualized geometry:** renders micro-polygon detail by streaming only the detail needed per pixel, so highly detailed Megascans/CAD-derived meshes run without hand-built LODs. This keeps triangle cost bounded, which matters because VR renders each frame twice (one per eye).
- **Occlusion culling:** skips rendering objects hidden behind walls or other geometry. In an indoor walkthrough most rooms are not visible at any moment, so culling cuts draw calls and GPU work sharply and stabilizes frame time.
- (Alternative: **LOD** swaps distant meshes for lower-poly versions, saving roughly 40–60% GPU cost at distance; **Instanced Static Mesh** merges repeated props like chairs into one draw call.) Verify with stat fps / stat GPU / Unreal Insights.

Q21 (b) Cross-platform MR safety-training simulation

1. Why Unreal suits building and scaling it (vs Unity)

- **Blueprints:** trainers and safety experts can prototype scenarios (alarms, hazards, step-by-step procedures) without C++; faster iteration.
- **Native C++:** performance-critical systems (physics-heavy interactions, many simultaneous hazards) can be optimized for PC-tethered headsets and still scaled down for a standalone Quest.
- **Built-in networking/replication framework:** supports multi-trainee sessions and instructor monitoring without third-party packages (Unity needs Netcode for GameObjects).
- **OpenXR support and scalability settings:** one project can target PC VR (SteamVR/Meta Link) and Quest (Android), and extend to HoloLens-class devices; Chaos physics gives realistic fire, impact and destruction.
- Unity comparison: strong for mobile/HoloLens (MRTK) and quick prototypes, but Unreal has the advantage in fidelity and built-in tooling for high-end PC; the studio may keep Quest quality by using LOD, instancing and lighter lighting.

2. Present limitations, future direction, two optimizations to plan

- **Limitations:** heavy GPU load and thermal limits on standalone headsets, fragmented device SDKs, large package sizes, expensive manual content creation, latency when streaming without edge compute.
- **Future direction:** cross-device standardization (OpenXR), cloud/edge (6G) rendering that offloads Unreal GPU work, AI-assisted content creation (MetaHuman, LLM avatars), haptics, spatial computing.
- **Optimization 1:** design for OpenXR from day one so the same app runs across PC, Quest and later HoloLens with minimal rework.

- **Optimization 2:** plan a cloud/edge-rendering path and AI-assisted asset/NPC generation so scenarios can scale without a bigger local GPU or large art team.

Q22 (a) VR physiotherapy on Quest

1. Geometric, kinematic and physical modeling applied

- **Geometric:** exercise objects (balls, cones, resistance bands, targets) as low-poly triangle meshes with PBR textures and simple collider shapes; correct real-world scale (1 unit = 1 m) so reaching distances are accurate; LOD to hold 72 fps.
- **Kinematic:** patient's head and hands tracked in 6 DoF; scene graph parents objects to the hand when grabbed; inverse kinematics estimates elbow, shoulder and spine so the avatar shows reaching and squatting; joint angles and range of motion can be measured without forces.
- **Physical:** Rigidbody with mass, drag and gravity, colliders and physic materials (bounciness/friction) so objects respond realistically when touched or pushed; broad-phase and narrow-phase collision detection; controller haptic pulses on contact.

2. Locomotion and Oculus configuration

- **Locomotion:** primarily **room-scale walking** (highest comfort) with **teleportation** for any larger movement; **snap turn** instead of smooth turning; avoid smooth thumbstick movement. Seated mode for patients who cannot stand.
- **Guardian:** set a clear, obstacle-free boundary slightly larger than the exercise reach; re-draw it each session; warn visually near the edge. This prevents collisions during reaching and squatting.
- **Interaction:** Grab/Grip interactable with Direct interactor for near objects, big targets with visual glow, audio and haptic feedback; hand tracking optional for patients who cannot hold controllers; passthrough option for safety awareness.
- **Comfort:** stable 72+ fps, tunnelling vignette, adjustable difficulty, height reset, short sessions.

Q22 (b) Metaverse virtual campus

1. Metaverse concept and behavior modeling

- **Metaverse:** persistent, shared, real-time network of 3D spaces where users interact as avatars, socialize and transact. Here: persistent campus, synchronous lectures, shared by Quest and browser users.
- **Behavior modeling (sense, decide, act):** avatars sense input (raise hand gesture, proximity, voice); rules/state machines decide (avatar states: idle, walking, seated, speaking); actions animate and update the shared world.
- **Objects:** event rules such as 'on grab/open then page-turn animation and show content', 'on hand raised then highlight student to instructor'; time-driven behavior (class schedule opens doors, lights dim for presentation).
- **NPCs:** guide or teaching-assistant NPC as an FSM with NavMesh (Idle, Greet, Guide, Explain), optionally LLM-driven to answer FAQs; instructor's position drives audio and focus.

2. Audio/video and Quest UI/UX together

- **Spatial 3D audio:** Audio Source with Spatial Blend = 1, logarithmic rolloff and HRTF spatializer so the instructor's voice comes from their avatar's position; reverb zones for hall acoustics; ITD/ILD/HRTF give direction and distance cues.
- **360 recorded lecture video:** Video Player → Render Texture → panoramic skybox for field-trip or lab recordings; note it is 3 DoF; use virtual cinema screen (quad) for slides; pair with ambisonic audio rotated to head orientation.
- **UI/UX:** diegetic/spatial UI: wrist menu, world-space panels at 1–3 m within ±30 degrees, large high-contrast text; poke/ray interaction; visual, audio and haptic feedback; avoid screen-fixed HUD.
- **Cross-device:** browser users get 2D versions of the same controls; seated mode, subtitles and comfort settings give an inclusive, usable experience.

Q23 (a) VR boxing / fitness on Quest

1. Kinematics and physical modeling for punches

- **Kinematics:** track controller (or hand) pose in 6 DoF every frame; compute punch velocity and direction from position change over time (p += v dt relationship); IK drives the avatar's arms; scene-graph parenting keeps gloves attached to hands; use Time.deltaTime so timing is consistent at

72/90/120 fps.

- **Physical:** gloves and opponent body parts have colliders; **Continuous collision detection** prevents fast punches tunnelling through the target; relative velocity and mass decide impact strength; Rigidbody forces/joints make the opponent or bag react (recoil, sway); haptic pulses on hit scaled by force; FixedUpdate physics step matched to headset refresh.
- **Opponent reactions** combine physics response with behavior (FSM: guard, attack, dodge, stagger).

2. UI/UX and audio design

- **UI:** world-space heart-rate/calorie dashboard as a **diegetic/spatial** element: a wrist display or a panel on the wall of the virtual gym placed 1.5–2 m away, slightly below eye level, curved, lazy-follow or world-locked, large high-contrast numbers; no screen-fixed HUD (breaks immersion and causes discomfort).
- **Audio:** spatial 3D audio (HRTF) for coach voice and opponent footsteps/punch sounds so direction and distance are believable; beat/rhythm cues with consistent timing; **ambisonic** ambient gym/crowd bed that rotates with head orientation for presence.
- Justification: immersion and clarity during intense motion, minimal need to look away, haptic/visual/audio feedback for each hit.

Q23 (b) Seated motion-platform roller coaster

1. Geometric and kinematic modeling

- **Geometric:** track defined as splines/curves (NURBS or spline-based), converted to a polygon mesh with rails, supports and scenery; correct metric scale; LOD and baked lighting/occlusion culling for 72+ fps; PBR materials for realism.
- **Kinematic:** camera/cart moves along the spline with position, velocity and orientation (pitch, yaw, roll) computed from the track's curve; speed profile set by script (no forces needed); the camera is a child of the cart in the scene graph; the same pose stream (position/rotation, time-based with Time.deltaTime) is sent to the motion platform so virtual and physical motion are synchronized; vehicle/cockpit as a stable reference.

2. Oculus setup/UI, ambisonic + spatial audio

- **Setup and UI:** seated/stationary Guardian boundary; minimal UI (no HUD; essential prompts only, diegetic such as on the cart); stable frame rate; vignette during sharp turns; seat belts/comfort settings; short sessions.
- **Sickness prevention:** because the real platform moves with the virtual camera, sensory conflict (vection without inner-ear motion) is reduced; avoid sudden unexpected acceleration or user-uncontrolled camera rotation beyond the platform's matching motion.
- **Audio:** pre-recorded **ambisonic** bed (wind, crowd, ambience) rotates with head orientation for presence; live **3D spatial** effects (clanks, screams, nearby objects) positioned by HRTF; both together anchor the user in the scene and support orientation, reducing disorientation.

Part 6 – Last-Night Revision Sheet

Keyword to answer map

Hear this	Say this
non-programmers / visual scripting	Unreal Blueprints
performance-critical language	C++ (Unity = C#)
real-time dynamic lighting	Lumen
huge detailed meshes, no manual LOD	Nanite
skip hidden objects	Occlusion culling
many small meshes, one call	Batching / instancing
distance-based simpler meshes	LOD
frame time bottleneck	Profiler: stat fps, stat GPU, stat unit
cross-device standard	OpenXR
shape and appearance	Geometric modeling
motion without forces	Kinematics (FK/IK, 6 DoF, 4x4 matrix)
mass, friction, collision response	Physical modeling
react, decide, NPC	Behavior modeling (FSM, NavMesh)
instant move, less sickness	Teleportation
safe play boundary	Guardian
pick up object	Grab / Grip interactable
UI in 3D scene	World-space / diegetic UI
directional sound	Spatial audio (ITD, ILD, HRTF)
rotatable surround sound	Ambisonic audio
persistent shared 3D worlds	Metaverse

Numbers to remember

Item	Value
VR target frame rate	90 fps (Unit 4); Quest labs 72+ fps; refresh options 72/90/120 Hz
LOD saving	About 40–60% GPU at distance; LOD1 50% tris, LOD2 25%
Unreal licence	5% royalty above $1M
Chaos lab	Voronoi cells 30, damage threshold 100, field strength 500000, radius 200 cm
Unreal install	32 GB RAM, RTX 2060, 80 GB SSD
Unity ARCore	Min API 24
UI placement	1–3 m, ±30 degrees, nothing reading-heavy under 0.5 m
Canvas scale	0.001 (1000 px = 1 m), 1.5 m ahead
Snap turn	30 or 45 degrees; move speed about 2 m/s
Hand joints	26 per hand
Physics step	0.02 s default FixedUpdate; gravity 9.81 m/s2
Ball example	Scale 0.22, mass 0.43, bounciness 0.6
XR pipeline rates	Depth 30 Hz, physics 120 Hz, display 90 fps

Viva-style questions (Unit 5 slides)

- Differentiate geometric, kinematic, physical and behavior modeling. • Why do GPUs render triangles? What is LOD?
- Why use 4x4 homogeneous matrices? • FK vs IK and where IK is used in VR. • Broad vs narrow phase. • What does Is Kinematic change?
- Teleportation vs smooth locomotion for comfort. • Steps to configure Unity for Meta Quest. • Interactor vs interactable; hover, select, activate.

- Diegetic vs spatial UI with one example each. • How ITD, ILD and HRTF create spatial audio. • Draw an FSM for a museum guide NPC.

Final tips: (1) Spend the first 20 minutes on Part A; 14 of 20 are Unit 5. (2) In Part B always label sub-parts 1 and 2 and tie each point to the scenario. (3) Mention frame rate, comfort and immersion in every VR answer. (4) If unsure on MCQs, the option that mentions the full definition (shape + surface + structure; forces + mass + friction; reacts over time to events) is usually right.

Detailed Supplement

Deep-dive explanations, worked numericals, extra scenario answers and 60 more MCQs

This supplement goes with Parts 1–6. It adds **why and how** explanations (what the examiner's L4 'analyze' questions need), worked numerical examples, extra likely scenario questions with full answers, short notes and a bigger MCQ bank.

Part	Content
7	Unit 4 deep dive (Unreal concepts, Blueprints vs C++, Nanite/Lumen limits, frame budgets, optimization workflow, Chaos, Unity AR Foundation)
8	Unit 5 deep dive (modeling theory, worked FK / matrix / physics numericals, locomotion, interaction, UI, audio, metaverse, behavior)
9	Six extra scenario questions with full 10-mark answers
10	Short notes (2–5 marks) and comparison tables
11	60 extra MCQs with answers

Part 7 – Unit 4 Deep Dive

7.1 Unreal Engine building blocks

- **Level (map):** the scene file containing everything placed in the world.
- **Actor:** any object placed in a level (cube, light, camera, character). Actors are built from **Components** (static mesh, collision, physics body, motion controller).
- **Blueprint:** a visual script / class made of nodes connected by wires in an **Event Graph**. Events (e.g. Event BeginPlay, Grip Pressed, On Hit) start execution; wires carry execution flow and data.
- **Play modes:** Play in editor (desktop), **VR Preview** (headset connected). **Packaging** builds an executable for Windows or an APK/OBB for Quest.
- **Plugins:** features are enabled in Edit → Plugins (OpenXR, Chaos Physics, Geometry Collection). Restart is required after enabling.

7.2 Blueprints vs C++

Aspect	Blueprints	C++
Who uses it	Designers, artists, rapid prototypers	Programmers
Speed of iteration	Very fast, no compile of the whole project	Slower: compile, hot reload
Runtime performance	Slower for heavy per-frame logic (interpreted nodes)	Fastest; native code
Best for	Gameplay logic, interaction, prototypes, VR grab/throw	Performance-critical systems, core engine extensions, large algorithms
Practice	Combine: core systems in C++, expose to Blueprints for designers	

Exam line: Unreal gives a two-tier workflow: Blueprints for rapid, designer-friendly iteration and C++ for performance-critical systems. Unity uses C# for everything and has no equivalent built-in primary visual scripting workflow.

7.3 Lumen and Nanite: what they solve, and their limits

	Lumen	Nanite
Solves	Lighting: realistic global illumination (bounced light) and reflections that update live	Geometry: very high triangle counts without manual LOD authoring
Old approach	Baked lightmaps (slow to rebuild, static)	Hand-made LOD chain, normal-map baking, strict poly budgets
How	Dynamic GI with soft shadows, no baking; light, time of day and materials can change live	Virtualized micro-polygon geometry; streams and refines detail per pixel automatically
Benefit in VR/archviz	Clients change lighting/materials live in meetings	Import film-quality scanned or CAD assets directly
Limit to mention	GPU heavy; best on high-end PC-tethered VR	Heavy for standalone headsets; Quest projects usually rely on LOD, instancing and baked lighting

Smart point for 21(b): Nanite and Lumen suit the PC-tethered headset; for the standalone Quest the same project needs scalability (LOD, instanced meshes, simplified lighting). That is exactly the 'present limitation' the question asks for.

7.4 Frame budgets and how to find the bottleneck

Refresh rate	Frame time budget (1000 / fps)
72 Hz	13.9 ms
90 Hz	11.1 ms
120 Hz	8.3 ms

In VR every frame is rendered **twice** (one per eye) and must never drop, or the user feels judder and cybersickness. Everything (CPU game logic, physics, draw submission, GPU rendering) must fit inside the budget.

Optimization workflow (say this order in answers)

- **1. Measure first:** stat fps (frame rate), stat unit (game/draw/GPU time breakdown), stat GPU (GPU passes), Unreal Insights for detailed traces. Unity: Profiler, Frame Debugger, GPU Usage.
- **2. Identify the bottleneck:** if GPU time is highest the scene is GPU-bound (too many pixels/triangles/lights); if Draw/Game time is highest it is CPU-bound (too many draw calls, heavy scripts, physics).
- **3. Apply the matching fix:** GPU-bound → LOD, Nanite, occlusion culling, cheaper materials/lighting. CPU-bound → draw-call batching, Instanced Static Mesh, simpler collision, fewer ticking actors.
- **4. Re-measure** and package a Development build first, then Shipping for release.

Worked example: draw calls

A hall has 200 identical chairs. Naively each chair is its own draw call = 200 draw calls per eye-pass. Converting them to an **Instanced Static Mesh** sends the chair mesh once with 200 transforms = 1 draw call. CPU submission cost falls sharply, and the saved time goes to the frame budget. The same idea in Unity is GPU Instancing / Static Batching.

LOD worked example

LOD0 = full mesh (100% triangles) close up; LOD1 at 50% triangles for mid-distance; LOD2 at 25% far away. A chair 20 m away drawn with LOD2 costs about a quarter of the triangles with no visible difference, which is the source of the 40–60% GPU saving across a whole scene.

Occlusion culling in detail

- Static (precomputed): the scene is analysed offline to decide which objects can see each other; at runtime the camera's cell lookup quickly drops hidden objects.
- Dynamic (hardware query): the GPU tests whether an object's bounding box is hidden behind already-drawn depth; costs some work but handles moving objects.
- Best for indoor scenes (rooms, corridors). Little benefit in wide open terrain.
- Related but different: **frustum culling** removes objects outside the camera's view cone; occlusion culling removes objects inside the cone that are blocked.

7.5 Chaos Physics deeper

- **Simple vs complex collision:** simple collision uses boxes/spheres/capsules for fast physics (collisions, trace). Complex collision uses the real triangle mesh for accurate line traces/precise hits but is costly. Use simple collision for moving physics objects.
- **Pipeline each physics tick:** apply forces → integrate velocities/positions → broadphase (cheap bounding-volume overlap) → narrowphase (exact contact points) → contact resolution (impulses, friction, restitution). Physics runs at a fixed rate (the slide gives 120 Hz) independent of rendering.
- **Destruction workflow:** pick a wall mesh → Fracture mode with Voronoi (Uniform, 30 cells) → Geometry Collection asset (each fragment is a piece) → Damage Threshold decides how much impulse breaks it → Radial Falloff Field applies outward force/strain when the ball hits → fragments simulate as rigid bodies.
- **Cost warning:** more cells = more rigid bodies = more CPU. Destruction in VR must be limited (fragment count, lifetime) to keep 90 fps.
- **Cloth:** per-panel stiffness, damping and wind on a skeletal mesh; use sparingly because it is simulated every frame.

7.6 Unity AR Foundation concepts

- **AR Foundation** is Unity's cross-platform AR API that wraps **ARCore** (Android) and **ARKit** (iOS). One C# project, two platforms.
- **AR Session:** runs the AR lifecycle and tracking. **AR Session Origin** (XR Origin in newer versions) holds the AR Camera and converts tracked positions into Unity world space; the default Main Camera is removed.
- **AR Plane Manager:** detects horizontal/vertical surfaces and spawns plane objects (the blue mesh on the floor). **AR Raycast Manager:** casts a ray from a touch point onto detected planes and returns a *pose* (position + rotation).
- **Tap-to-place logic:** on touch → Raycast(touch position, hits, TrackableType.Planes) → if hit, Instantiate(prefab, hits[0].pose.position, hits[0].pose.rotation).

- **Why Unity for mobile AR:** low-overhead rendering, easy phone deployment, AR Foundation abstraction, large asset ecosystem. Unreal can do AR but is heavier for mobile GPUs.
- **Build checks:** switch platform to Android, Minimum API Level 24, tick ARCore in XR Plug-in Management, USB debugging on, Build And Run.

7.7 Unreal vs Unity: decision guide

Project need	Pick	Reason
Photoreal archviz walkthrough, high-end PC VR	Unreal	Lumen + Nanite + Blueprints
Mobile AR furniture placement	Unity	AR Foundation, ARCore/ARKit, light on mobile GPU
HoloLens medical training	Unity	MRTK hand tracking, voice, spatial anchors
AAA game / film virtual production	Unreal	Fidelity, C++, Chaos, Megascans
Quick indie XR prototype	Unity	C#, Asset Store, fast iteration
Designer-driven iteration without coding	Unreal	Blueprints
Cross-device future (PC, Quest, HoloLens)	Either	Both support OpenXR; plan scalability

Part 8 – Unit 5 Deep Dive and Worked Numericals

8.1 Geometric modeling: why each choice matters

- **Why polygon meshes in VR?** GPUs are built to rasterize triangles very fast. NURBS/CSG are converted (meshed) before real-time use.
- **Stereo cost:** two eye views per frame, so triangle and pixel cost is roughly doubled. Standalone headsets have mobile-class GPUs, so geometry must be budgeted tightly.
- **Normal maps** fake detail with texture data so triangle count stays low. **LOD** reduces triangles with distance. **Decimation** reduces triangles of scanned meshes.
- **Bounding volume** (AABB, sphere) is a cheap approximate shape used for quick collision and culling tests.
- **Scale matters:** physical and kinematic models assume real units (1 Unity unit = 1 m). Wrong scale breaks gravity feel, reach distance, audio rolloff and UI size.

8.2 Kinematics: worked numericals

Forward kinematics (2-link arm)

Given L1 = 2 m, L2 = 1.5 m, θ1 = 30°, θ2 = 45° (second joint angle relative to first link):

Step	Calculation	Result
x	L1 cos θ1 + L2 cos(θ1 + θ2) = 2 cos 30° + 1.5 cos 75°	1.732 + 0.388 = 2.12 m
y	L1 sin θ1 + L2 sin(θ1 + θ2) = 2 sin 30° + 1.5 sin 75°	1.000 + 1.449 = 2.45 m

FK has exactly one answer. IK would start from the target (2.12, 2.45) and search for θ1, θ2; it can have two solutions (elbow up / elbow down), one, or none if the target is out of reach (distance greater than L1 + L2 = 3.5 m).

Homogeneous transform (4x4) example

A local point p_local = (1, 0, 0). Object is rotated 90° about the vertical (Y) axis, then translated by (2, 0, 3). Rotation takes (1,0,0) to (0,0,−1); adding the translation gives **p_world = (2, 0, 2)**. In matrix form p_world = T · R · S · p_local: scale is applied first, rotation second, translation last (matrices multiply right to left). Using 4x4 matrices lets rotation and translation be combined into one matrix so a chain of parents is a single multiplication.

Scene graph example

Spanner is a child of Right Controller. When grabbed, M_world(spanner) = M_world(controller) · M_local(spanner). Moving the hand moves the spanner without extra code. Releasing re-parents it to the world (or enables physics).

Kinematic vs dynamic objects in Unity

- **Is Kinematic ticked:** object is moved by script, animation or tracking; physics does not push it, but it still pushes other (non-kinematic) bodies. Example: a door animated on a hinge, a hand/controller proxy.
- **Not kinematic:** forces, gravity and collisions control it. Example: a thrown ball.

8.3 Physical modeling: worked numericals

One physics step

Object mass m = 2 kg, net force F = 10 N, time step Δt = 0.02 s, starting at rest.

Step	Formula	Value
Acceleration	a = F / m	10 / 2 = 5 m/s²
Velocity	v += a·Δt	0 + 5×0.02 = 0.10 m/s
Position	x += v·Δt	0 + 0.10×0.02 = 0.002 m

Repeating this 50 times (1 second) gives the continuous motion. With gravity only (a = 9.81 m/s²) the ball reaches about 9.81 m/s after 1 s. **Restitution (bounciness)** e scales the rebound speed: a ball hitting at 5 m/s with e = 0.6 rebounds at about 3 m/s.

Why Continuous collision detection?

A fast ball (say 20 m/s) at 90 fps moves 0.22 m per frame, which is more than its own diameter (0.22 m football), so a discrete check could skip over a thin wall (**tunnelling**). Continuous collision detection sweeps the shape along its path to catch the hit. It is why the throwable ball example uses Continuous.

Hand-held object physics

- **Velocity Tracking:** the held object is driven by physics velocities toward the hand pose, so it still collides with walls.
- **Instantaneous / Kinematic tracking:** object snaps to the hand with no collisions (can pass through walls).
- **Throw On Detach:** on release the hand's velocity is transferred so the throw feels natural.

Haptics

Controller vibration amplitude/duration can be scaled by impact speed (same relativeVelocity idea as the impact sound). Bare-hand tracking has no haptics, so compensate with visual glow and audio. Force-feedback gloves (HaptX, Ultraleap) are the future trend in Unit 4.

8.4 Locomotion in depth

Why cybersickness happens

- **Sensory conflict:** the eyes report motion (vection) while the vestibular system (inner ear) reports none.
- **Triggers:** artificial rotation (strongest), acceleration/deceleration, low or unstable frame rate, high latency (motion-to-photon), wide field of view moving, camera moved without user control.
- **Mitigations:** teleportation, snap turn, tunnelling vignette, stable reference frame (cockpit/horizon), constant speed, 72–120 Hz, user comfort options.

Choosing locomotion for a scenario

Scenario	Best technique	Why
Rehabilitation / physiotherapy	Room-scale + teleport, snap turn, seated mode	Safe, natural body movement; no sickness
Large virtual campus	Teleport with anchors	Cover 2 km from a 3 m room comfortably
Roller coaster / vehicle	Vehicle/cockpit (seated)	Stable reference, matches motion platform
Fast action game for experienced users	Smooth movement + vignette + comfort options	Immersive but needs settings
Climbing / rock wall	Grab-move / climbing	Natural hand-driven movement
Large empty gym	Redirected walking	Subtle rotation keeps user inside the room (needs space)

8.5 Object interaction in depth

- **XR Interaction Manager** mediates between interactors and interactables, deciding who can hover/select.
- **Interaction states: Hover** (pointing/near, highlight) → **Select** (grip, picks up) → **Activate** (trigger, uses the object: switch torch on).
- **Direct vs Ray vs Poke vs Socket:** Direct for touching distance; Ray for far objects and panels; Poke for fingertip buttons; Socket for snapping objects into slots (key into lock).
- **Attach Transform:** defines where on the object the hand holds it (torch handle).
- **Controllers vs hand tracking:** controllers give buttons and haptics; hand tracking is natural but has no haptics and less precision (use pinch/poke/grab with large targets).

8.6 UI/UX checklist for exam answers

- World-space (diegetic/spatial) UI, 1–3 m, within ±30° of straight ahead and slightly below eye level, curved panels.
- Large, high-contrast text; avoid head-locked UI; use lazy-follow or world-lock.
- Multi-sense feedback (visual + audio + haptic).
- Comfort/accessibility: seated mode, height reset, subtitles, one-handed play, colour-blind-safe cues.
- Minimal UI in seated/motion experiences; essential info only; place on the vehicle or wrist.

8.7 Audio and video in depth

- **Why spatial audio matters:** directional and distance cues let users locate sources, increase presence and give eyes-free guidance (e.g. coaching cues behind you).
- **Human cues:** ITD (time difference, up to roughly 0.6 ms between ears) for left/right at low frequencies, ILD (level difference) for high frequencies, HRTF (ear/head/torso filtering) for elevation and front/back.
- **Distance cues:** volume rolloff, high-frequency loss, reverb ratio. **Reverb zones** give room acoustics.
- **Ambisonic audio:** records the whole sound sphere in a format that is rotated to match head orientation; ideal for 360 video ambience. Combine with object-based 3D sources for interactive sounds.
- **360 video:** mono (flat), stereo 360 (depth), 180 stereo (sharper front half), volumetric (walk-around). Only 3 DoF; leaning does not change view. Use hardware-decoded H.264/H.265 and high resolution to avoid blur and stutter.
- **Unity pipeline:** Video Player → Render Texture → Skybox/Panoramic material (or quad for a virtual cinema screen).

8.8 Metaverse in depth

- **Technologies:** VR/AR/MR headsets, 3D engines, avatars, real-time networking (Netcode for GameObjects), cloud/edge, AI agents, digital twins.
- **Architecture for a shared world:** server holds persistent world state; clients render locally and sync avatar poses, object states and voice. Persistence = state saved on server; synchronous = low-latency updates.
- **Challenges:** interoperability across platforms, privacy and data (body tracking, eye tracking), latency and bandwidth, cost of hardware, safety/harassment and moderation, digital identity, accessibility.
- **Seven layers (Radoff):** Experience, Discovery, Creator economy, Spatial computing, Decentralization, Human interface, Infrastructure (top to bottom: 7 to 1).
- **Real-world check:** consumer hype faded (Reality Labs losses about $80B; Horizon Worlds moving mobile-first), while industrial digital twins, training and mixed reality keep growing.

8.9 Behavior modeling in depth

- **FSM:** finite set of states, one active at a time, transitions triggered by conditions. Simple, predictable, easy to debug; can become messy with many states.
- **Behaviour tree:** hierarchical tasks with priorities (selectors, sequences); scales better than FSM for complex NPCs.
- **Utility AI:** scores each action and picks the best (pet chooses eat/play/sleep).
- **Reinforcement learning (ML-Agents):** learns from rewards; **LLM-driven:** plans and talks (tutor, receptionist).
- **Pseudo-code for a shopkeeper:** sense (user near? stock low?) → decide (if user near: Greet; if question asked: Answer; if stock low: Restock; else Idle) → act (animate, speak, move with NavMesh).
- **Multi-user behavior:** avatars' poses are synchronized over the network; object ownership rules decide who controls a shared object.

Part 9 – Six Extra Scenario Questions with Full Answers

These follow the same style as the sample paper. Practice writing each in 25–30 minutes.

E1 (Unit 4). VR demolition-safety training with destruction and physics

A company builds a VR simulator where trainees knock down a wall with a swinging hammer. (1) Explain how Chaos physics models the wall breaking. (2) Suggest optimizations to keep 90 fps.

1. Chaos physics modeling

- **Rigid body dynamics:** hammer and fragments are rigid bodies with mass and damping; swing impulse = mass x velocity.
- **Collision detection:** simple collision shapes for hammer and fragments; broadphase prunes pairs, narrowphase finds contact points, contact resolution applies impulse, friction and bounce.
- **Chaos Destruction:** wall fractured in Fracture mode using Voronoi (e.g. 30 cells) into a Geometry Collection; Damage Threshold (e.g. 100) decides when a piece breaks; a Radial Falloff Field (strength, radius) applies outward strain on hit.
- **Feedback:** controller haptics scaled by impact force; spatial audio for crashes.

2. Optimizations

- Limit fragment count and lifetime; swap small debris to simple collision or remove after a few seconds.
- Use LOD and Instanced Static Mesh for repeated site props; occlusion culling for indoor areas.
- Profile with stat unit/stat GPU/Unreal Insights; keep physics tick at fixed rate; test on target headset in Development then Shipping build.

E2 (Unit 4). Mobile AR furniture-placement app: which engine?

A retailer wants phone-based AR where customers place furniture in their room. (1) Choose between Unreal and Unity. (2) Explain the AR pipeline and key steps.

1. Choice: Unity with AR Foundation

- Unity's AR Foundation wraps ARCore and ARKit so one C# project targets Android and iOS; low-overhead rendering suits mobile GPUs. IKEA Place and Pokémon GO-style overlays are Unity examples.
- Unreal is stronger for high-fidelity PC/console/archviz; heavier for mobile; so Unity is the better fit here.

2. Pipeline and steps

- Sensors (camera, IMU, depth) → ARCore/ARKit tracking and 6DoF SLAM, point cloud → engine places objects at anchors → frame composited over camera feed on the phone display; touch input feeds back.
- Setup: install AR Foundation + ARCore plugin, Android build settings (min API 24), AR Session + Session Origin, AR Plane Manager, script with raycast to plane and Instantiate prefab, Build And Run.

E3 (Unit 5). VR museum with 360 video, spatial audio and UI

(1) Describe how geometric modeling and kinematics produce the museum's exhibits and doors. (2) Describe audio/video and UI design for visitors using Quest.

1. Geometric and kinematic

- **Geometric:** exhibits scanned via photogrammetry/LiDAR, decimated, UV-unwrapped, normal-map baked, exported FBX/glTF, imported at 1 unit = 1 m with LOD Group and baked occlusion; PBR materials.
- **Kinematic:** doors/lifts animated via localRotation/Slerp or Animator using Time.deltaTime; scene graph parents grabbed artifacts to the hand; IK for guide avatar arms.

2. Audio/video and UI

- Guide's voice as 3D spatial audio (Spatial Blend 1, HRTF, rolloff) so visitors hear direction; reverb zones in halls; ambisonic ambience for 360 field-trip clips.
- 360 videos shown on a skybox/panoramic material via Video Player + Render Texture (3 DoF), or a virtual-cinema quad for flat video.
- UI: world-space labels above exhibits (spatial UI), wrist menu map, panels 1–3 m away within ±30°, large text; ray for far labels, poke for near; teleport anchors at rooms with snap turn and vignette;

seated mode and subtitles.

E4 (Unit 5). VR driving simulator

(1) Apply kinematic and physical modeling to the car. (2) Choose locomotion/UI choices to reduce sickness.

1. Modeling

- **Kinematic:** steering wheel and pedals as hinged/rotating parts (localRotation), driver camera as a child of the car in the scene graph, hands via IK to the wheel.
- **Physical:** Rigidbody for the car with mass, drag, friction (wheel colliders), suspension via joints/springs, collision with road/obstacles, restitution on crashes, FixedUpdate physics, controller haptics for road feel.

2. Comfort

- Vehicle/cockpit locomotion gives a **stable reference frame** (dashboard, windscreen frame); avoid sudden acceleration; steady 72–90+ fps; vignette in sharp turns; seated Guardian/stationary boundary; seat height reset; diegetic UI on the dashboard instead of HUD; optional motion platform; 3D engine audio from engine/road with spatial HRTF.

E5 (Unit 5). AI tutor NPC in a virtual classroom

(1) Explain behavior modeling for the tutor. (2) Explain the metaverse context.

1. Behavior modeling

- Sense-decide-act: senses student proximity, gaze/hand raised, spoken question; decides with an FSM (Idle, Greet, Explain, Answer, Guide) or behaviour tree; LLM-driven component generates answers; acts by animating, speaking in spatial audio, walking with NavMeshAgent to the student's desk.
- Level 4 (intelligent) behavior: learns or converses using AI; level 3 (agent) adds goals and states.

2. Metaverse context

- Persistent, shared, synchronous classroom accessible from Quest and browser; avatars embodied; tutor NPC is part of the experience layer; relies on infrastructure (cloud, 5G/6G), human interface (headsets, haptics), decentralization (edge/AI agents) and spatial computing (3D engine, physics).
- Challenges: privacy, latency, moderation, interoperability, accessibility.

E6 (Unit 5). VR surgical training

(1) Explain how geometric, kinematic and physical modeling combine. (2) Explain interaction and haptics.

1. Combined modeling

- **Geometric:** anatomy from CT/MRI voxels or scans converted to meshes; high detail with normal maps; accurate scale.
- **Kinematic:** tracked hand/tool pose (6 DoF), tool is a child of the hand, IK for arms.
- **Physical:** soft-tissue deformation (soft body), collision between tool and tissue, friction, resistance forces (bone-cutting resistance in Osso VR example), joints for instruments.

2. Interaction and haptics

- Grab/Direct interactors for instruments with attach transforms; trigger activates tool; Socket for returning tools to tray.
- Haptic gloves/controllers produce force or vibration according to simulated contact force; audio and visual cues; Velocity Tracking so tools collide with tissue; stable high frame rate is essential for hand–eye accuracy.

Part 10 – Short Notes and Comparison Tables

Short notes (answer in 4–5 lines)

Blueprints: Unreal's node-based visual scripting. Nodes and wires in an Event Graph let non-programmers create gameplay logic without C++; designers iterate quickly. Combined with C++ for performance-critical code.

Lumen: Unreal 5's fully dynamic global illumination and reflection system. No lightmap baking, soft shadows, supports live light and day/night changes. Best on high-end hardware.

Nanite: Virtualized micro-polygon geometry in UE5. Lets film-quality meshes (Megascans) render in real time without manual LODs; streams detail on the fly.

Chaos Physics: Unreal's physics engine: rigid bodies, collision (broad/narrow phase), cloth, and destruction using Voronoi fracture and fields.

OpenXR: Open Khronos standard API that lets one app run across many headsets (Quest, Vive, Index, HoloLens 2, WMR). Key for cross-device standardization.

Occlusion culling: Skips rendering objects hidden behind others; reduces GPU load, especially in indoor scenes.

Draw call batching / instancing: Combines many small meshes or instances into few GPU calls; reduces CPU submission cost (Instanced Static Mesh, GPU Instancing).

Pose and 6 DoF: Position (X,Y,Z) plus orientation (pitch, yaw, roll). Quest tracks all six; phone VR tracks 3.

Scene graph: Hierarchy of parent-child transforms; child world matrix = parent world matrix x local matrix.

Forward vs inverse kinematics: FK: angles to position (one answer). IK: target position to angles (many/no answers, numerical). IK drives full-body avatars from head+hands.

Broad / narrow phase: Broad phase prunes far-apart pairs using bounding volumes; narrow phase runs exact tests (SAT, GJK) to get contact points and depth.

Cybersickness: Nausea/discomfort from sensory conflict between visual motion and inner-ear sensing; reduced by teleport, snap turn, vignette, stable frame rate.

Guardian boundary: User-drawn safe play area on Quest that prevents collisions with real obstacles during stationary or room-scale play.

Diegetic UI: UI that exists inside the 3D world (wristwatch, dashboard, tablet); best for immersion. Spatial UI floats in the world; non-diegetic HUD is avoided.

HRTF: Head-related transfer function: filtering by head/ear/torso that lets the brain judge elevation and front/back; used by spatializers in VR audio.

Ambisonic audio: Full-sphere surround format that rotates with head orientation; used with 360 video.

Metaverse: Persistent, shared, real-time 3D worlds with avatars, creation, work and trade; seven-layer value chain.

Finite state machine: Behavior model with fixed states and condition-based transitions, one state active at a time (guide: Idle, Greet, Guide, Explain).

Comparison tables often asked

Teleportation	Smooth locomotion
Instant jump to a point; high comfort	Continuous glide; low-to-medium comfort (vection)
Loses spatial/motion continuity	Preserves continuity, feels natural to gamers
Good for large worlds, beginners, rehab	Good for fast action with comfort options

Kinematics modeling	Physical modeling
Describes motion (position, velocity, angles)	Describes why motion happens (forces, mass, friction)
No forces involved	Forces, collisions, restitution, deformation
Examples: scripted door, IK arm, animation	Examples: thrown ball, destruction, cloth

Spatial audio	Ambisonic audio
Object-based 3D sources positioned in the scene	Scene-based full-sphere recording
Moves with source and listener, uses HRTF	Rotated to listener head orientation
Voices, effects, interactive sounds	Ambience for 360 video

FSM	Behaviour tree
Few states, simple, predictable	Hierarchical, prioritized tasks
Gets tangled as states grow	Scales to complex NPCs
Guard patrol/chase/return	Shopkeeper serving queues

Part 11 – 60 Extra MCQs

1. Which engine feature streams micro-polygon detail automatically? → Nanite
2. Which language do Unreal Blueprints replace for designers? → C++ scripting for gameplay logic
3. A scene is GPU-bound. Best first fix? → LOD / Nanite / occlusion culling / cheaper materials
4. A scene is CPU-bound by draw calls. Best fix? → Instanced Static Mesh / batching
5. Frame time budget at 90 Hz? → About 11.1 ms
6. Frame time budget at 72 Hz? → About 13.9 ms
7. Which plugin must be enabled for Chaos destruction? → Chaos Physics + Geometry Collection
8. Fracture method used in the lab? → Voronoi (Uniform), 30 cells
9. What does the Damage Threshold control? → How much impulse fractures the object (100 in lab)
10. What applies outward force to fragments? → Radial Falloff Field / Apply External Strain
11. Which command shows frame rate in Unreal? → stat fps
12. Which Unreal tool gives detailed performance traces? → Unreal Insights
13. Unity's equivalent of LOD tool? → LOD Group component
14. Unity's occlusion culling is based on? → Umbra baking
15. Which SDK wraps ARCore/ARKit in Unity? → AR Foundation
16. What does AR Plane Manager do? → Detects surfaces and creates plane objects
17. Which AR script call finds the tap position on a floor? → Raycast against TrackableType.Planes
18. Packaging output for Quest? → APK (plus OBB)
19. Which build configuration first? → Development, then Shipping
20. Which headset runs Horizon OS? → Meta Quest (Android-based)
21. Which processor is in Quest 3 per slides? → Snapdragon XR2 Gen 2
22. Which budget Quest is in the 2026 lineup? → Quest 3S
23. What is a pose? → Position and orientation of an object
24. Rotation about the vertical axis is called? → Yaw
25. Nodding 'yes' is which rotation? → Pitch
26. Tilting head sideways is? → Roll
27. Bottom row of a 4x4 transform matrix? → 0 0 0 1
28. What does p_world = T R S p_local mean? → Scale first, then rotate, then translate
29. IK has how many solutions? → Many, one or none
30. FK is easy because? → Joint angles give exactly one position
31. Why use Slerp? → Smooth interpolation between two rotations
32. Why multiply by Time.deltaTime? → Frame-rate independent speed
33. Physics step in Unity runs in? → FixedUpdate
34. Restitution value of 1 means? → Perfect bounce
35. Is Kinematic ticked means? → Not moved by forces; moved by script
36. Collider that is cheapest? → Box / Sphere / Capsule
37. Why Continuous collision detection for fast balls? → Prevents tunnelling through thin objects
38. Which interaction state corresponds to grip press? → Select
39. Which state corresponds to trigger press? → Activate
40. Which interactor presses buttons with fingertip? → Poke
41. Which interactor holds objects in a slot? → Socket
42. Hand tracking gesture to select far objects? → Pinch with a ray
43. Hand tracking lacks which feedback? → Haptic vibration
44. Which UI is part of the story and in the 3D world? → Diegetic
45. Which UI should be avoided in VR? → Non-diegetic HUD glued to the screen

46. Wrist menu shows when? → Palm turns toward the face
47. Canvas render mode for VR menu? → World Space
48. Which raycaster for XR UI? → Tracked Device Graphic Raycaster
49. ITD stands for? → Interaural Time Difference
50. ILD stands for? → Interaural Level Difference
51. HRTF helps judge? → Elevation and front/back
52. Spatial Blend for full 3D? → 1
53. 360 stereo vs mono? → Stereo adds depth (two views stacked)
54. Does 360 video allow leaning to change view? → No: 3 DoF
55. Metaverse term origin? → Snow Crash (Neal Stephenson, 1992)
56. Layer 1 of metaverse? → Infrastructure (5G/6G, cloud GPUs)
57. Layer 7 of metaverse? → Experience
58. Which behavior level is a door opening as you approach? → Level 1 reactive
59. NavMesh is used for? → Path-finding on walkable areas
60. Which AI technique learns from rewards in Unity? → Reinforcement learning (ML-Agents)
61. Netcode in Unity for multi-user? → Netcode for GameObjects

Study plan for tonight: (1) Read Part 6 sheet and Part 10 short notes (30 min). (2) Do Section A of the sample paper without looking, then check Part 4 (20 min). (3) Write one Unit 4 answer (Q21) and two Unit 5 answers (Q22, Q23) from memory using the template (60 min). (4) Re-read worked numericals in Part 8 (15 min). (5) Sleep; do not add new topics.